

NASM 32-bit Assembly

Worked Examples

Module 1: Getting Started with NASM 32-bit

Purpose. Use these examples as models for tracing registers, memory, directives, system calls, and algorithms before attempting the separate student workbook exercises.

Build workflow. Every normal source file uses ``nasm -f elf32``, ``ld -m elf_i386``, then the generated executable. Command-line investigation examples include their own extra commands.

Important. Study the pattern and explanation first. The values, messages, filenames, and scenarios intentionally differ from the exercise set so students still need to solve the assigned problems themselves.

Example 1: Course Welcome Message

Related concept. Write, assemble, link, and run the canonical Hello World program, and confirm that the student can execute the complete build workflow on Ubuntu.

Example task. Practice the same NASM technique in a different context: course welcome message.

Filename. m01_example_01.asm

Step-by-Step Procedure

1. Encode the ASM program.
2. Assemble it with: ``nasm -f elf32 m01_example_01.asm -o m01_example_01.o``.
3. Link it with: ``ld -m elf_i386 -o m01_example_01 m01_example_01.o``.
4. Run with ``./m01_example_01``
5. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Key Source Pattern

```
SYS_EXIT equ 1
SYS_WRITE equ 4
STDOUT   equ 1

section .text
    global _start
_start:
    ; write the greeting to standard output
    mov eax, SYS_WRITE          ; syscall number 4
    mov ebx, STDOUT             ; file descriptor 1 (screen)
    mov ecx, message            ; address of the string
    mov edx, message_len        ; number of bytes to write
    int 0x80                    ; trigger the system call

    ; exit cleanly
    mov eax, SYS_EXIT
    xor ebx, ebx                ; exit code 0
    int 0x80
```

```
section .data
    message db 'Welcome to Assembly Lab!', 0xa    ; 0xa = newline
    message_len equ $ - message    ; length of the message bytes
```

Expected Output

```
Welcome to Assembly Lab!
```

Explanation

The related course objective is to write, assemble, link, and run the canonical Hello World program, and confirm that the student can execute the complete build workflow on Ubuntu. Trace the program sections, the `\_start` entry point, and the Linux `int 0x80` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDI gives the byte count.

Example 2: Return a Custom Exit Status

Related concept. Demonstrate that a program returns a numeric status to the shell and that the shell can read it with `echo $?`.

Example task. Practice the same NASM technique in a different context: return a custom exit status.

Filename m01_example_02.asm

Step-by-Step Procedure

1. Encode the asm program.
2. Assemble it with: ``nasm -f elf32 m01_example_02.asm -o m01_example_02.o``.
3. Link it with: ``ld -m elf_i386 -o m01_example_02 m01_example_02.o``.
4. Run it with: ``./m01_example_02``.
5. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Key Source Pattern

```
SYS_EXIT equ 1

section .text
    global _start
_start:
    mov eax, SYS_EXIT        ; syscall number 1
    mov ebx, 12              ; exit with status 12
    int 0x80
```

Expected Output

```
$ ./m01_example_02
$ echo $?
```

Explanation

The related course objective is to demonstrate that a program returns a numeric status to the shell and that the shell can read it with `echo $?`. Trace the program sections, the `_start` entry point, and the Linux `int 0x80` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDX gives the byte count.

Example 3: Print Two Laboratory Announcements

Related concept. Print two separate strings with two independent `sys_write` calls, confirming that each call prints exactly the bytes it was given.

Example task. Practice the same NASM technique in a different context: print two laboratory announcements.

Filename. `m01_example_03.asm`

Step-by-Step Procedure

1. Encode the asm program
2. Assemble it with: ``nasm -f elf32 m01_example_03.asm -o m01_example_03.o``.
3. Link it with: ``ld -m elf_i386 -o m01_example_03 m01_example_03.o``.
4. Run it with: ``./m01_example_03``.
5. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Key Source Pattern

```
SYS_EXIT equ 1
SYS_WRITE equ 4
STDOUT equ 1

section .text
    global _start
_start:
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    mov ecx, line1
    mov edx, len_line1
    int 0x80

    mov eax, SYS_WRITE
    mov ebx, STDOUT
    mov ecx, line2
    mov edx, len_line2
    int 0x80

    mov eax, SYS_EXIT
    xor ebx, ebx
    int 0x80

section .data
; ... open the accompanying .asm file for the complete program ...
```

Expected Output

```
Lab starts at 8:00 AM.  
Bring your NASM worksheet.
```

Explanation

The related course objective is to print two separate strings with two independent `sys_write` calls, confirming that each call prints exactly the bytes it was given. Trace the program sections, the `__start` entry point, and the Linux `int 0x80` calling convention. For output, `EAX` selects the system call, `EBX` carries the file descriptor, `ECX` points to the buffer, and `EDX` gives the byte count.

Example 4: Display a Three-Line Book Record

Related concept. Personalize a three-line output program with the student's own name, course, and section.

Example task. Practice the same NASM technique in a different context: display a three-line book record.

Filename. 01_example_04.asm

Step-by-Step Procedure

1. Encode the asm program.
2. Assemble it with: `nasm -f elf32 m01_example_04.asm -o m01_example_04.o`.
3. Link it with: `ld -m elf_i386 -o m01_example_04 m01_example_04.o`.
4. Run it with: `./m01_example_04`.
5. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Key Source Pattern

```
SYS_EXIT equ 1  
SYS_WRITE equ 4  
STDOUT equ 1  
  
section .text  
global _start  
_start:  
    mov eax, SYS_WRITE  
    mov ebx, STDOUT  
    mov ecx, myname  
    mov edx, len_myname  
    int 0x80  
  
    mov eax, SYS_WRITE  
    mov ebx, STDOUT  
    mov ecx, mycourse  
    mov edx, len_mycourse  
    int 0x80  
  
    mov eax, SYS_WRITE  
    mov ebx, STDOUT  
    mov ecx, mysection  
    mov edx, len_mysection  
    int 0x80  
  
    mov eax, SYS_EXIT
```

```

xor ebx, ebx
int 0x80

section .data
myname db 'Computer Architecture', 0xa
len_myname equ $ - myname
mycourse db 'Author: M. Rivera', 0xa
len_mycourse equ $ - mycourse
mysection db 'Shelf: CS-04', 0xa
len_mysection equ $ - mysection

```

Expected Output

```

Computer Architecture
Author: M. Rivera
Shelf: CS-04

```

Explanation

The related course objective is to personalize a three-line output program with the student's own name, course, and section. Trace the program sections, the `\_start` entry point, and the Linux `int 0x80` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDI gives the byte count.

Example 5: Repair a Broken Status Message

Related concept. Diagnose and repair a program that assembles and links but prints nothing, finding all three bugs.

Example task. Practice the same NASM technique in a different context: repair a broken status message.

Filename. m01_example_05.asm

Step-by-Step Procedure

1. Encode the asm program.
2. Assemble it with: `nasm -f elf32 m01\_example\_05.asm -o m01\_example\_05.o`.
3. Link it with: `ld -m elf\_i386 -o m01\_example\_05 m01\_example\_05.o`.
4. Run it with: `./m01\_example\_05`.
5. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Key Source Pattern

```

SYS_EXIT equ 1
SYS_WRITE equ 4
STDOUT equ          ; FIX 3:

section .text
global _hadji          ; FIX 1:
_start:
mov eax, SYS_WRITE
mov ebx, STDOUT          ; file descriptor

```

```

mov ecx, message          ; FIX 2: buffer address goes in ECX, not EBX
mov edx, message_len
int 0x80

mov eax, SYS_EXIT
mov ebx, 0
int 0x80

section .data
message2 db 'Print repaired!', 0xa
message_len equ $ - message

```

Expected Output

```
Print repaired!
```

Explanation

The related course objective is to diagnose and repair a program that assembles and links but prints nothing, finding all three bugs. Trace the program sections, the `\_start` entry point, and the Linux `int 0x80` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDX gives the byte count.

Example 6: Compare Program and Shell Exit Codes

Related concept. Use the shell to observe how programs communicate results through their exit status, without writing any new code.

Example task. Practice the same NASM technique in a different context: compare program and shell exit codes.

Filename. 01_example_06.asm

Step-by-Step Procedure

1. Encode the asm program.
2. Assemble it with: `nasm -f elf32 m01\_example\_06.asm -o m01\_example\_06.o`.
3. Link it with: `ld -m elf\_i386 -o m01\_example\_06 m01\_example\_06.o`.
4. Run: `./m01\_example\_06; echo \$?`.
5. Run: `false; echo \$?`.
6. Run: `true; echo \$?`.
7. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Note. This is a command-line example. The ASM program only returns status 23; the shell commands compare it with standard success/failure statuses.

Key Source Pattern

```
SYS_EXIT equ 1

section .text
    global _start
_start:
    mov eax, SYS_EXIT
    mov ebx, 23          ; custom status for this example
    int 0x80
```

Expected Output

```
$ ./m01_example_06; echo $?
23
$ false; echo $?
1
$ true; echo $?
0
```

Explanation

The related course objective is to use the shell to observe how programs communicate results through their exit status, without writing any new code. Trace the program sections, the `\_start` entry point, and the Linux `int 0x80` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDX gives the byte count.

Example 7: Inspect the Machine Bytes of a Write Program

Related concept. Use the ndisasm disassembler to view the actual instruction bytes the assembler produced.

Example task. Practice the same NASM technique in a different context: inspect the machine bytes of a write program.

Filename. 01_example_07.asm

Step-by-Step Procedure

1. Encode the asm program.
2. Assemble it with: `nasm -f elf32 m01\_example\_07.asm -o m01\_example\_07.o`.
3. Link it with: `ld -m elf\_i386 -o m01\_example\_07 m01\_example\_07.o`.
4. Run: `nasm -f elf32 m01\_example\_07.asm -o m01\_example\_07.o`.
5. Run: `objdump -d -M intel m01\_example\_07.o | head -20`.
6. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Note. Exact addresses/relocation bytes may differ, but the instruction opcodes and int 0x80 bytes are the focus.

Key Source Pattern

```
SYS_EXIT equ 1
```

```

SYS_WRITE equ 4
STDOUT    equ 1

section .text
    global _start
_start:
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    mov ecx, message
    mov edx, message_len
    int 0x80

    mov eax, SYS_EXIT
    xor ebx, ebx
    int 0x80

section .data
    message db 'BYTE CHECK', 0xa
    message_len equ $ - message

```

Expected Output

```

Example excerpt:
B8 04 00 00 00    mov eax,0x4
BB 01 00 00 00    mov ebx,0x1
...
CD 80             int 0x80

```

Explanation

The related course objective is to use the `ndisasm` disassembler to view the actual instruction bytes the assembler produced. Trace the program sections, the `_start` entry point, and the Linux `int 0x80` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDX gives the byte count.

Example 8: Inspect a Small 32-bit ELF Executable

Related concept. Inspect the structure of the linked executable with `objdump`, `size`, and `file`, and relate the sections of the binary to the sections of the source.

Example task. Practice the same NASM technique in a different context: inspect a small 32-bit elf executable.

Filename. `01_example_08.asm`

Step-by-Step Procedure

1. Open the accompanying source file `m01_example_08.asm` and identify the section(s), constants, labels, and registers used by the example.
2. Assemble it with: `nasm -f elf32 m01_example_08.asm -o m01_example_08.o`.
3. Link it with: `ld -m elf_i386 -o m01_example_08 m01_example_08.o`.
4. Run: `nasm -f elf32 m01_example_08.asm -o m01_example_08.o`.
5. Run: `ld -m elf_i386 -o m01_example_08 m01_example_08.o`.
6. Run: `file m01_example_08`.

7. Run: ``size m01_example_08``.
8. Run: ``objdump -d -M intel m01_example_08 | head -25``.
9. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Note. Numeric section sizes depend on the exact toolchain and linker version, so compare structure rather than requiring one exact byte count.

Key Source Pattern

```
SYS_EXIT equ 1
SYS_WRITE equ 4
STDOUT equ 1

section .text
    global _start
_start:
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    mov ecx, message
    mov edx, message_len
    int 0x80

    mov eax, SYS_EXIT
    xor ebx, ebx
    int 0x80

section .data
    message db 'ELF CHECK', 0xa
    message_len equ $ - message
```

Expected Output

The file command identifies a 32-bit LSB ELF executable.
size reports text/data/bss sizes.
objdump shows the machine instructions with executable addresses.

Explanation

The related course objective is to inspect the structure of the linked executable with objdump, size, and file, and relate the sections of the binary to the sections of the source. Trace the program sections, the ``_start`` entry point, and the Linux ``int 0x80`` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDX gives the byte count.

Example 9: Compare Block Output with Byte-at-a-Time Output

Related concept. Compare printing a string with a single `sys_write` call against printing it one byte at a time in a loop.

Example task. Practice the same NASM technique in a different context: compare block output with byte-at-a-time output.

Filename. m01_example_09.asm

Step-by-Step Procedure

1. Encode the asm program.
2. Assemble it with: ``nasm -f elf32 m01_example_09.asm -o m01_example_09.o``.
3. Link it with: ``ld -m elf_i386 -o m01_example_09 m01_example_09.o``.
4. Run it with: ``./m01_example_09``.
5. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Key Source Pattern

```
SYS_EXIT equ 1
SYS_WRITE equ 4
STDOUT equ 1

section .text
    global _start
_start:
    ; Approach A: one big sys_write
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    mov ecx, whole
    mov edx, len_whole
    int 0x80

    ; Approach B: many small sys_write calls (one byte each)
    mov ecx, parts
    mov edx, 1
    mov dword [i], 0
b:
    mov al, byte [i]                ; load current index (as byte)
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    push ecx                        ; save the buffer start across the call
    mov ecx, parts
    add ecx, [i]                    ; point at the i-th character
    int 0x80
    pop ecx
    inc dword [i]
    cmp dword [i], len_parts
    jle b

    mov eax, SYS_EXIT
    xor ebx, ebx
    int 0x80

section .data
    whole db 'Block write. ', 0
    len_whole equ $ - whole - 1
    parts db 'Byte writes.', 0
    len_parts equ $ - parts - 1

section .bss
    i resd 1
```

Expected Output

Block write. Byte writes.

Explanation

The related course objective is to compare printing a string with a single `sys_write` call against printing it one byte at a time in a loop. Trace the program sections, the `_start` entry point, and the Linux `int 0x80` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDX gives the byte count.

Example 10: Print Five Computer Specifications with Procedures

Related concept. Use the first procedures — five one-line print helpers — to organize a program that prints five labeled personal details.

Example task. Practice the same NASM technique in a different context: print five computer specifications with procedures.

Filename. `m01_example_10.asm`

Step-by-Step Procedure

1. Open the accompanying source file `m01_example_10.asm` and identify the section(s), constants, labels, and registers used by the example.
2. Assemble it with: `nasm -f elf32 m01_example_10.asm -o m01_example_10.o`.
3. Link it with: `ld -m elf_i386 -o m01_example_10 m01_example_10.o`.
4. Run it with: `./m01_example_10`.
5. Compare the result with the expected output, then change one safe data value or message and rebuild to observe what changes and what stays the same.

Key Source Pattern

```
SYS_EXIT equ 1
SYS_WRITE equ 4
STDOUT equ 1

section .text
    global _start
_start:
    call print_name
    call print_id
    call print_course
    call print_school
    call print_motto

    mov eax, SYS_EXIT
    xor ebx, ebx
    int 0x80

; helper: ECX = buffer, EDX = length -> sys_write
print_it:
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    int 0x80
    ret

print_name:
    mov ecx, name_line
    mov edx, len_name_line
    jmp print_it
```

```

print_id:
    mov ecx, id_line
    mov edx, len_id_line
    jmp print_it

print_course:
    mov ecx, course_line
    mov edx, len_course_line
    jmp print_it

print_school:
    mov ecx, school_line
    mov edx, len_school_line
    jmp print_it

print_motto:
    mov ecx, motto_line
    mov edx, len_motto_line
    jmp print_it

section .data
    name_line    db 'Device:  Lab PC 07', 0xa
    len_name_line equ $ - name_line
    id_line       db 'CPU:      Core i5', 0xa
    len_id_line   equ $ - id_line
    course_line   db 'Memory:  16 GB', 0xa
    len_course_line equ $ - course_line
    school_line   db 'Storage: 512 GB SSD', 0xa
    len_school_line equ $ - school_line
    motto_line    db 'OS:       Ubuntu Linux', 0xa
    len_motto_line equ $ - motto_line

```

Expected Output

```

Device:  Lab PC 07
CPU:     Core i5
Memory:  16 GB
Storage: 512 GB SSD
OS:      Ubuntu Linux

```

Explanation

The related course objective is to use the first procedures — five one-line print helpers — to organize a program that prints five labeled personal details. Trace the program sections, the `\_start` entry point, and the Linux `int 0x80` calling convention. For output, EAX selects the system call, EBX carries the file descriptor, ECX points to the buffer, and EDX gives the byte count.